데이크스트라 스터디

2026년 3월 5일

중량제한(1939)

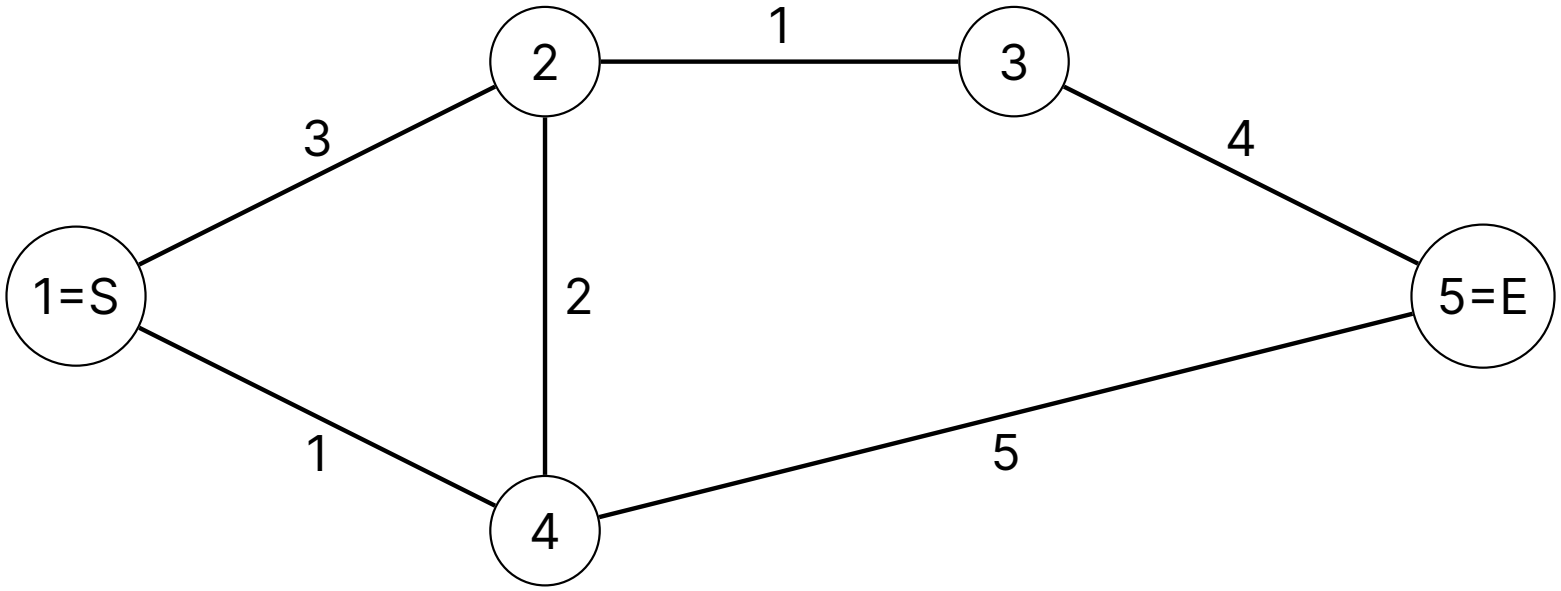
문제

$N(2 \le N \le 10{,}000)$개의 섬으로 이루어진 나라가 있다. 이들 중 몇 개의 섬 사이에는 다리가 설치되어 있어서 차들이 다닐 수 있다.
영식 중공업에서는 두 개의 섬에 공장을 세워 두고 물품을 생산하는 일을 하고 있다. 물품을 생산하다 보면 공장에서 다른 공장으로 생산 중이던 물품을 수송해야 할 일이 생기곤 한다. 그런데 각각의 다리마다 중량제한이 있기 때문에 무턱대고 물품을 옮길 수 없다. 만약 중량제한을 초과하는 양의 물품이 다리를 지나게 되면 다리가 무너지게 된다.
한 번의 이동에서 옮길 수 있는 물품들의 중량의 최댓값을 구하는 프로그램을 작성하시오.

입력

첫째 줄에 N, $M(1 \le M \le 100{,}000)$이 주어진다. 다음 M개의 줄에는 다리에 대한 정보를 나타내는 세 정수 A, $B(1 \le A, B \le N)$, $C(1 \le C \le 1{,}000{,}000{,}000)$가 주어진다. 이는 A번 섬과 B번 섬 사이에 중량제한이 C인 다리가 존재한다는 의미이다.
서로 같은 두 섬 사이에 여러 개의 다리가 있을 수도 있으며, 모든 다리는 양방향이다.
마지막 줄에는 공장이 위치해 있는 섬의 번호를 나타내는 서로 다른 두 정수가 주어진다. 공장이 있는 두 섬을 연결하는 경로는 항상 존재하는 데이터만 입력으로 주어진다.

예시 그래프 G



데이크스트라 풀이

일반적인 가중치 있는 그래프에서 경로의 가중치는 간선들의 가중치의 합입니다. 즉 $w_1 + w_2 + \cdots + w_k$ 입니다.

이 문제에서는 가장 작은 간선의 가중치가 경로의 가중치입니다. 즉 $\min(w_1, w_2, \ldots, w_k)$ 입니다. 따라서, 경로의 길이를 구하는 방법을 min 연산으로 바꿔서 데이크스트라 알고리즘을 적용할 수 있습니다.

정의찬 데이크스트라 소스 코드 1

```
1   import java.io.*;
2   import java.util.*;
3
4   class Dijkstra {
5       public static void main(String[] args) throws NumberFormatException,
        ↪   IOException {
6           BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
7           StringBuilder sb = new StringBuilder();
8           StringTokenizer st = new StringTokenizer(br.readLine());
9
10          int n = Integer.parseInt(st.nextToken());
11          int m = Integer.parseInt(st.nextToken());
12
13          List<int[]>[] nexts = new List[n + 1];
14          for (int i = 1; i <= n; i++)
15              nexts[i] = new ArrayList<>();
16
17          for (int i = 0; i < m; i++) {
18              st = new StringTokenizer(br.readLine());
19              int u = Integer.parseInt(st.nextToken());
20              int v = Integer.parseInt(st.nextToken());
21              int w = Integer.parseInt(st.nextToken());
22              nexts[u].add(new int[] { v, w });
23              nexts[v].add(new int[] { u, w });
24          }
25
26          st = new StringTokenizer(br.readLine());
27          int s = Integer.parseInt(st.nextToken());
28          int e = Integer.parseInt(st.nextToken());
```

정의찬 데이크스트라 소스 코드 2

```java

        PriorityQueue<int[]> q = new PriorityQueue<>((o1, o2) -> Integer.compare(o2[1], o1[1]));
        int[] visited = new int[n + 1];
        q.add(new int[] { s, Integer.MAX_VALUE });
        visited[s] = Integer.MAX_VALUE;

        while (!q.isEmpty()) {
            int[] current = q.poll();
            if (current[0] == e)
                break;
            for (int[] next : nexts[current[0]]) {
                int nextW = next[1] > current[1] ? current[1] : next[1];

                if (visited[next[0]] < nextW) {
                    visited[next[0]] = nextW;

                    q.add(new int[] { next[0], nextW });
                }
            }
        }

        System.out.println(visited[e]);
    }
}
```

서민종 데이크스트라 1 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int N, M;
    static List<Bridge>[] bridges;
    static long[] capacities;
    static int start, end;
    static boolean[] visited;

    static class Bridge {
        int end;
        long capacity;

        public Bridge(int end, long capacity) {
            this.end = end;
            this.capacity = capacity;
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new
        ↪  OutputStreamWriter(System.out));
        StringTokenizer st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());
        bridges = new List[N + 1];
        for (int i = 1; i <= N; i++) {
```

서민종 데이크스트라 1 소스 코드 2

```
29                bridges[i] = new ArrayList<>();
30            }
31            for (int i = 0; i < M; i++) {
32                st = new StringTokenizer(br.readLine());
33                int A = Integer.parseInt(st.nextToken());
34                int B = Integer.parseInt(st.nextToken());
35                long C = Long.parseLong(st.nextToken());
36                putInBridgeList(A, B, C);
37            }
38            st = new StringTokenizer(br.readLine());
39            start = Integer.parseInt(st.nextToken());
40            end = Integer.parseInt(st.nextToken());
41            dijkstra();
42            bw.write(capacities[end] + "\n");
43            bw.flush();
44        }
45
46        public static void dijkstra() {
47            capacities = new long[N + 1];
48            PriorityQueue<Bridge> pq = new PriorityQueue<>(new Comparator<Bridge>() {
49                @Override
50                public int compare(Bridge o1, Bridge o2) {
51                    return (int) (o2.capacity - o1.capacity);
52                }
53            });
54            visited = new boolean[N + 1];
55            for (Bridge b : bridges[start]) {
56                pq.offer(b);
57                capacities[b.end] = b.capacity;
```

서민종 데이크스트라 1 소스 코드 3

```
        }
        visited[start] = true;
        while (!pq.isEmpty()) {
            Bridge b = pq.poll();
            int current = b.end;
            visited[current] = true;
            if (current == end) {
                return;
            }
            for (Bridge nb : bridges[current]) {
                int next = nb.end;
                long nc = Math.min(nb.capacity, capacities[current]);
                if (!visited[next] && capacities[next] < nc) {
                    capacities[next] = nc;
                    pq.offer(nb);
                }
            }
        }
    }

    public static void putInBridgeList(int A, int B, long C) {
        int index = -1;
        for (int j = 0; j < bridges[A].size(); j++) {
            if (bridges[A].get(j).end == B) {
                bridges[A].get(j).capacity = Math.max(bridges[A].get(j).capacity,
                ↪  C);
                index = j;
                break;
            }
```

서민종 데이크스트라 1 소스 코드 4

```
            }
            if (index == -1) {
                bridges[A].add(new Bridge(B, C));
            }
            index = -1;
            for (int j = 0; j < bridges[B].size(); j++) {
                if (bridges[B].get(j).end == A) {
                    bridges[B].get(j).capacity = Math.max(bridges[B].get(j).capacity,
                    ↪  C);
                    index = j;
                    break;
                }
            }
            if (index == -1) {
                bridges[B].add(new Bridge(A, C));
            }
        }
    }
```

서민종 데이크스트라 2 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main2 {
    static int N, M;
    static HashMap<Integer, Long>[] bridges;
    static Long[] maxCapacities;
    static int start, end;

    static class Bridge {
        int num;
        long capacity;

        public Bridge(int num, long capacity) {
            this.num = num;
            this.capacity = capacity;
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new
        ↪  OutputStreamWriter(System.out));
        StringTokenizer st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());
        bridges = new HashMap[N + 1];
        for (int i = 1; i <= N; i++) {
            bridges[i] = new HashMap<>();
```

서민종 데이크스트라 2 소스 코드 2

```
29          }
30          for (int i = 0; i < M; i++) {
31              st = new StringTokenizer(br.readLine());
32              int A = Integer.parseInt(st.nextToken());
33              int B = Integer.parseInt(st.nextToken());
34              long C = Long.parseLong(st.nextToken());
35              bridges[A].put(B, Math.max(bridges[A].getOrDefault(B, 0L), C));
36              bridges[B].put(A, Math.max(bridges[B].getOrDefault(A, 0L), C));
37          }
38          st = new StringTokenizer(br.readLine());
39          start = Integer.parseInt(st.nextToken());
40          end = Integer.parseInt(st.nextToken());
41          dijkstra();
42          bw.write(maxCapacities[end] + "\n");
43          bw.flush();
44      }
45
46      public static void dijkstra() {
47          maxCapacities = new Long[N + 1];
48          PriorityQueue<Bridge> pq = new PriorityQueue<>(new Comparator<Bridge>() {
49              @Override
50              public int compare(Bridge o1, Bridge o2) {
51                  if (o1.capacity > o2.capacity) {
52                      return -1;
53                  }
54                  return 1;
55              }
56          });
57          boolean[] visited = new boolean[N + 1];
```

서민종 데이크스트라 2 소스 코드 3

```
        for (int num : bridges[start].keySet()) {
            long capacity = bridges[start].get(num);
            pq.offer(new Bridge(num, capacity));
            maxCapacities[num] = capacity;
        }
        visited[start] = true;
        while (!pq.isEmpty()) {
            Bridge current = pq.poll();
            int cn = current.num;
            long cc = current.capacity;
            if (!visited[cn]) {
                visited[cn] = true;
                if (cn == end) {
                    return;
                }
                for (int nn : bridges[cn].keySet()) {
                    long nc = bridges[cn].get(nn);
                    if (!visited[nn]) {
                        if (maxCapacities[nn] == null || maxCapacities[nn] <
                        ↪  Math.min(cc, nc)) {
                            maxCapacities[nn] = Math.min(cc, nc);
                            pq.offer(new Bridge(nn, maxCapacities[nn]));
                        }
                    }
                }
            }
        }
    }
}
```

김준형 데이크스트라 소스 코드 1

```
1  import java.io.*;
2  import java.sql.Array;
3  import java.sql.ResultSet;
4  import java.sql.SQLException;
5  import java.util.*;
6
7  public class Main {
8      static class Cord implements Comparable<Cord> {
9          int n;
10         int w;
11
12         public Cord(int n, int w) {
13             this.n = n;
14             this.w = w;
15         }
16
17         @Override
18         public int compareTo(Cord c) {
19             return c.w - this.w;
20         }
21     }
22
23     static int n, m, s, e;
24     static int[] minTable;
25     static List<List<Cord>> graph = new ArrayList<>();
26     static PriorityQueue<Cord> que = new PriorityQueue<>();
27
28     public static void main(String[] args) throws IOException {
29         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

김준형 데이크스트라 소스 코드 2

```
30          StringTokenizer st = new StringTokenizer(br.readLine());
31          n = Integer.parseInt(st.nextToken());
32          m = Integer.parseInt(st.nextToken());
33
34          minTable = new int[n + 1];
35          for (int i = 0; i < n + 1; i++) {
36              graph.add(new ArrayList<>());
37          }
38
39          for (int i = 0; i < m; i++) {
40              st = new StringTokenizer(br.readLine());
41              int s = Integer.parseInt(st.nextToken());
42              int e = Integer.parseInt(st.nextToken());
43              int w = Integer.parseInt(st.nextToken());
44              graph.get(s).add(new Cord(e, w));
45              graph.get(e).add(new Cord(s, w));
46
47          }
48
49          st = new StringTokenizer(br.readLine());
50          s = Integer.parseInt(st.nextToken());
51          e = Integer.parseInt(st.nextToken());
52          minTable[s] = 1000000001;
53
54          for (int i = 0; i < graph.get(s).size(); i++) {
55              Cord cur = graph.get(s).get(i);
56              que.offer(cur);
57          }
58
```

김준형 데이크스트라 소스 코드 3

```
59          while (!que.isEmpty()) {
60
61              Cord cur = que.poll();
62              int curN = cur.n;
63              int curW = cur.w;
64
65             if (curW <= minTable[curN]) { // 이 다리까지 현재무게 보다 더 크게 가져왔다면 생략
66                  continue;
67              } else {
68                  minTable[curN] = curW; // 이 다리까지 현재무게가 최대치라면 갱신
69              }
70
71              for (int i = 0; i < graph.get(curN).size(); i++) {
72                  Cord next = graph.get(curN).get(i);
73                  int nextN = next.n;
74                  int nextW = next.w;
75
76                  int minW = Math.min(curW, nextW);
77                  if (minW > minTable[nextN]) {
78                      que.offer(new Cord(nextN, minW));
79                  }
80              }
81          }
82          System.out.print(minTable[e]);
83      }
84  }
```

김지훈 데이크스트라 소스 코드 1

```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.StringTokenizer;

public class BOJ_1939 {
    static int N, M, res, START, END;
    static int[] weight;
    static ArrayList<ArrayList<Edge>> adjList;

    public static void main(String[] args) throws Exception {

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        N = Integer.parseInt(st.nextToken()); // 정점
        M = Integer.parseInt(st.nextToken()); // 간선
        res = 0;

        adjList = new ArrayList<>();
        for (int i = 0; i <= N; i++) {
            adjList.add(new ArrayList<>());
        }

        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
```

김지훈 데이크스트라 소스 코드 2

```
30                int b = Integer.parseInt(st.nextToken());
31                int w = Integer.parseInt(st.nextToken());
32                adjList.get(a).add(new Edge(b, w));
33                adjList.get(b).add(new Edge(a, w));
34            }
35
36            st = new StringTokenizer(br.readLine());
37            START = Integer.parseInt(st.nextToken());
38            END = Integer.parseInt(st.nextToken());
39
40            weight = new int[N + 1];
41            Arrays.fill(weight, -1);
42            solve();
43
44            System.out.println(weight[END]);
45        }
46
47        private static void solve() {
48            PriorityQueue<Edge> pq = new PriorityQueue<>();
49            pq.add(new Edge(START, Integer.MAX_VALUE));
50            weight[START] = Integer.MAX_VALUE;
51
52            while (!pq.isEmpty()) {
53                Edge cur = pq.poll();
54                int c_v = cur.vertex;
55                int c_w = cur.value;
56
57                if (c_w < weight[c_v])
58                    continue;
```

김지훈 데이크스트라 소스 코드 3

```
59
60              for (Edge e : adjList.get(c_v)) {
61                  int next = e.vertex;
62                  int value = e.value;
63
64                  int temp = Math.min(weight[c_v], value);
65
66                  if (temp > weight[next]) {
67                      weight[next] = temp;
68                      pq.add(new Edge(next, weight[next]));
69                  }
70              }
71          }
72      }
73
74      static class Edge implements Comparable<Edge> {
75          int vertex, value;
76
77          Edge(int vertex, int value) {
78              this.vertex = vertex;
79              this.value = value;
80          }
81
82          @Override
83          public int compareTo(Edge e) {
84              return Integer.compare(e.value, this.value);
85          }
86      }
87  }
```

임건애 데이크스트라 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {

    static int N, M;
    static List<ArrayList<int[]>> graph;
    static int[] dist;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        StringTokenizer st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());

        graph = new ArrayList<>();
        for (int i = 0; i <= N; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());

            graph.get(a).add(new int[] { b, w });
            graph.get(b).add(new int[] { a, w });
```

임건애 데이크스트라 소스 코드 2

```
30          }
31
32          st = new StringTokenizer(br.readLine());
33          int start = Integer.parseInt(st.nextToken());
34          int end = Integer.parseInt(st.nextToken());
35
36          dist = new int[N + 1];
37          dijkstra(start);
38
39          System.out.println(dist[end]);
40      }
41
42      static void dijkstra(int start) {
43          PriorityQueue<int[]> pq = new PriorityQueue<>((o1, o2) -> o2[1] - o1[1]);
44          pq.offer(new int[] { start, Integer.MAX_VALUE });
45
46          dist[start] = Integer.MAX_VALUE;
47
48          while (!pq.isEmpty()) {
49              int[] current = pq.poll();
50              int cur = current[0], weight = current[1];
51
52              if (dist[cur] > weight)
53                  continue;
54
55              for (int[] next : graph.get(cur)) {
56                  int n = next[0], nw = next[1];
57
58                  int limit = Math.min(nw, dist[cur]);
```

임건애 데이크스트라 소스 코드 3

```
                    if (dist[n] < limit) {
                        dist[n] = limit;
                        pq.offer(new int[] { n, limit });
                    }

                }

            }

        }

    }
```

김시온 데이크스트라 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Dijkstra {

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        ArrayList<ArrayList<int[]>> adj = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            adj.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());

            adj.get(u).add(new int[] { v, w });
            adj.get(v).add(new int[] { u, w });
        }

        st = new StringTokenizer(br.readLine());
        int s = Integer.parseInt(st.nextToken());
        int e = Integer.parseInt(st.nextToken());
```

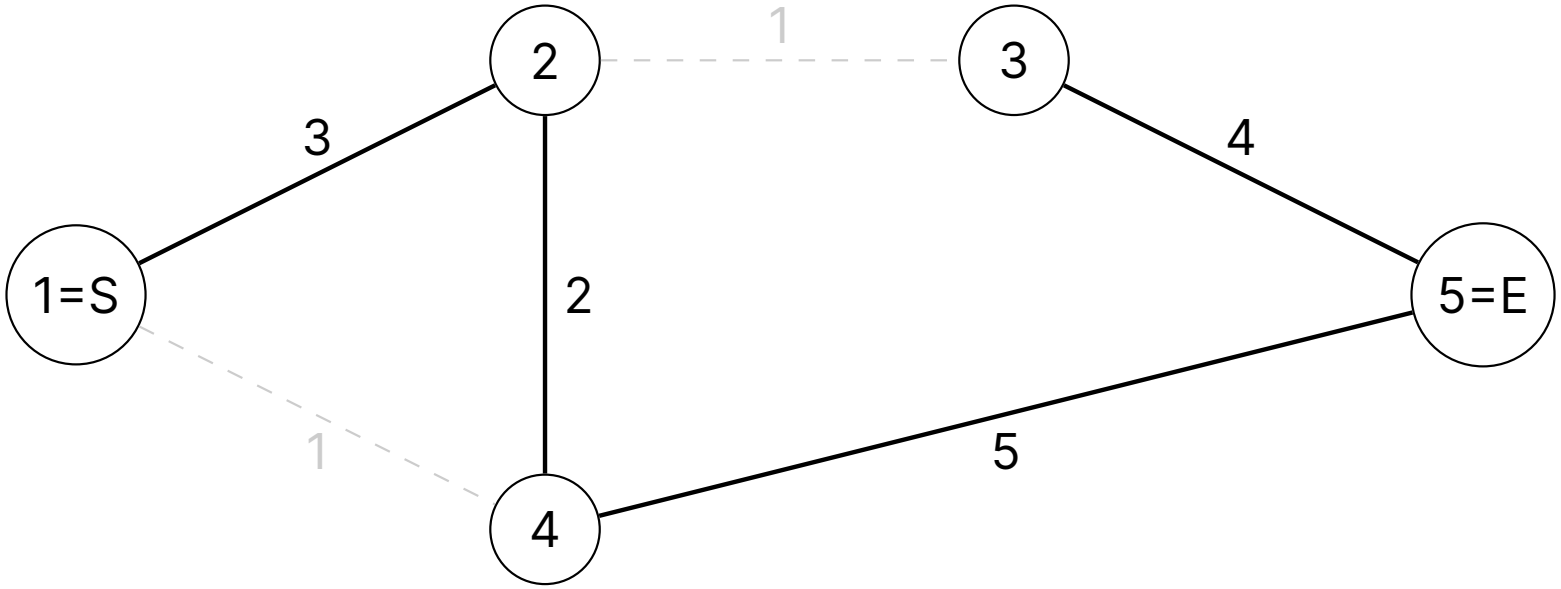
김시온 데이크스트라 소스 코드 2

```java

        PriorityQueue<int[]> q = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        int[] d = new int[n + 1];

        q.offer(new int[] { (int) 1e9, s });
        d[s] = (int) 1e9;

        while (!q.isEmpty()) {
            int[] t = q.poll();
            int c = t[0], u = t[1];

            if (d[u] != c) {
                continue;
            }

            for (int[] edge : adj.get(u)) {
                int v = edge[0], w = edge[1];

                int nw = Math.min(c, w);
                if (d[v] < nw) {
                    d[v] = nw;
                    q.add(new int[] { nw, v });
                }
            }
        }

        System.out.println(d[e]);
    }
}
```

부분 그래프 G_w

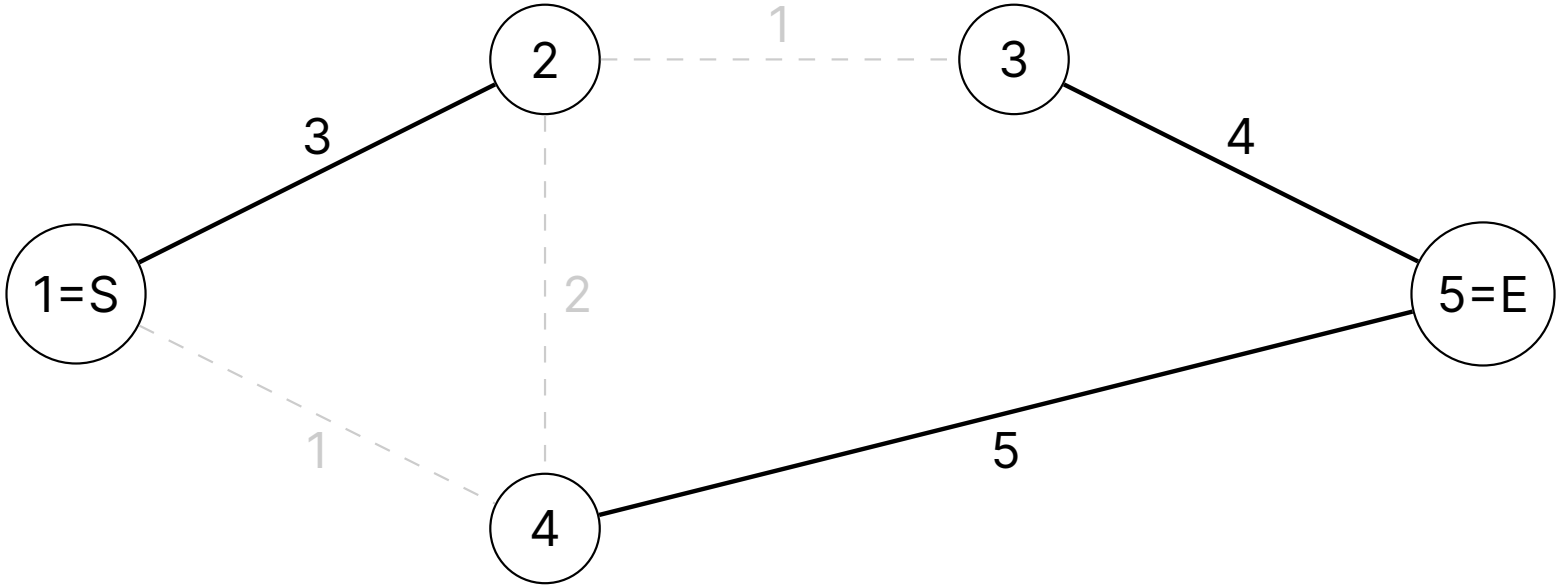
정의

G_w 는 그래프 G 의 모든 정점과 **가중치가 w 이상인 간선만** 포함하는 그래프입니다.



예를 들어 위 그래프에서 G_2 는 가중치가 2 이상인 간선만 남긴 그래프입니다.

G_3 예시



G_3 에서는 가중치가 3 이상인 간선만 남습니다.

G_w의 의미와 단조성

만약 G_w에서 S에서 E까지 경로가 존재한다면, **중량 w의 물품을 옮길 수 있습니다.**

판정 문제

G_w에서 S에서 E까지 경로가 존재하는가?

만약 $f(w)$가 참이라면 더 작은 가중치에서도 항상 참입니다.

$$f(w) = true \Rightarrow f(w-1) = true$$

따라서, $f(w)$는 다음과 같은 형태를 가집니다.

$$T, T, T, T, F, F, F$$

정의찬 이분 탐색 소스 코드 1

```
import java.io.*;
import java.util.*;

class BinarySearch {
    static int n;
    static List<int[]>[] nexts;
    static int s;
    static int e;

    public static void main(String[] args) throws NumberFormatException, IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        StringTokenizer st = new StringTokenizer(br.readLine());

        n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        nexts = new List[n + 1];
        for (int i = 1; i <= n; i++)
            nexts[i] = new ArrayList<>();

        int maxW = 0;

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());
```

정의찬 이분 탐색 소스 코드 2

```java

            nexts[u].add(new int[] { v, w });
            nexts[v].add(new int[] { u, w });
            maxW = Math.max(w, maxW);
        }

        st = new StringTokenizer(br.readLine());
        s = Integer.parseInt(st.nextToken());
        e = Integer.parseInt(st.nextToken());

        int ans = 0;
        int l = 1;
        int r = maxW;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (valid(mid)) {
                ans = mid;
                l = mid + 1;
            } else {
                r = mid - 1;
            }
        }
        System.out.println(ans);

    }

    static boolean valid(int min) {
        boolean[] visited = new boolean[n + 1];
        Queue<Integer> q = new LinkedList<>();
```

정의찬 이분 탐색 소스 코드 3

```
        q.add(e);
        visited[e] = true;

        while (!q.isEmpty()) {
            int current = q.poll();
            for (int[] next : nexts[current]) {
                if (!visited[next[0]] && (min <= next[1])) {
                    visited[next[0]] = true;
                    q.add(next[0]);
                }
            }
        }
        return visited[s];
    }
}
```

김시온 이분 탐색 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class BinarySearch {
    static int n, m;
    static ArrayList<ArrayList<int[]>> g = new ArrayList<>();
    static int s, e;

    static boolean f(int x) {
        Queue<Integer> q = new LinkedList<>();
        boolean[] b = new boolean[n + 1];

        q.add(s);
        b[s] = true;

        while (!q.isEmpty()) {
            int u = q.poll();

            for (int[] t : g.get(u)) {
                int v = t[0], w = t[1];

                if (w >= x && !b[v]) {
                    q.add(v);
                    b[v] = true;
                }
            }
        }

        return b[e];
```

김시온 이분 탐색 소스 코드 2

```
30      }
31
32      public static void main(String[] args) throws Exception {
33          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
34          StringTokenizer st = new StringTokenizer(br.readLine());
35          n = Integer.parseInt(st.nextToken());
36          m = Integer.parseInt(st.nextToken());
37
38          for (int i = 0; i <= n; i++) {
39              g.add(new ArrayList<>());
40          }
41
42          for (int i = 0; i < m; i++) {
43              st = new StringTokenizer(br.readLine());
44              int u = Integer.parseInt(st.nextToken());
45              int v = Integer.parseInt(st.nextToken());
46              int w = Integer.parseInt(st.nextToken());
47
48              g.get(u).add(new int[] { v, w });
49              g.get(v).add(new int[] { u, w });
50          }
51
52          st = new StringTokenizer(br.readLine());
53          s = Integer.parseInt(st.nextToken());
54          e = Integer.parseInt(st.nextToken());
55
56          int l = 1, r = (int) 1e9 + 1;
57          while (l + 1 < r) {
58              int mid = (l + r) / 2;
```

김시온 이분 탐색 소스 코드 3

```
            if (f(mid)) {
                l = mid;
            } else {
                r = mid;
            }
        }

        System.out.println(l);
    }
}
```

MST 풀이

무향 그래프에서 두 정점 사이의 경로 판별을 위해 그래프 탐색이나 유니온-파인드 자료구조를 사용할 수 있습니다.
S와 E가 연결될 때까지 가장 가중치가 큰 간선부터 추가하며 그래프를 구성합니다.
MST를 구하는 방법과 같습니다.
시간 복잡도는 $\mathcal{O}(M \log M)$입니다.

원찬혁 MST 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int n, m;
    static int p[];

    static int find(int v) {
        if (p[v] == 0)
            return v;
        p[v] = find(p[v]);
        return p[v];
    }

    static void merge(int a, int b) {
        int pa = find(a);
        int pb = find(b);
        if (pa == pb)
            return;
        p[pb] = pa;
    }

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        PriorityQueue<Node> q = new PriorityQueue<>();
        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        p = new int[n + 1];
```

원찬혁 MST 소스 코드 2

```
        int a, b, c, res = 0;
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            a = Integer.parseInt(st.nextToken());
            b = Integer.parseInt(st.nextToken());
            c = Integer.parseInt(st.nextToken());
            q.offer(new Node(c, a, b));
        }
        st = new StringTokenizer(br.readLine());
        a = Integer.parseInt(st.nextToken());
        b = Integer.parseInt(st.nextToken());
        while (find(a) != find(b)) {
            Node node = q.poll();
            res = node.w;
            merge(node.a, node.b);
        }
        System.out.println(res);
    }
}

class Node implements Comparable<Node> {
    int w;
    int a;
    int b;

    public Node(int w, int a, int b) {
        this.w = w;
        this.a = a;
        this.b = b;
```

원찬혁 MST 소스 코드 3

```
    }

    @Override
    public int compareTo(Node o) {
        return Integer.compare(o.w, this.w);
    }
}
```

김시온 MST 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class MST {
    static int[] p;

    static int find(int u) {
        if (p[u] == 0) {
            return u;
        }
        return p[u] = find(p[u]);
    }

    static void union(int u, int v) {
        u = find(u);
        v = find(v);
        if (u != v) {
            p[u] = v;
        }
    }

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        int[][] edge = new int[m][3];
        for (int i = 0; i < m; i++) {
```

김시온 MST 소스 코드 2

```
            st = new StringTokenizer(br.readLine());
            edge[i][0] = Integer.parseInt(st.nextToken());
            edge[i][1] = Integer.parseInt(st.nextToken());
            edge[i][2] = Integer.parseInt(st.nextToken());
        }

        st = new StringTokenizer(br.readLine());
        int s = Integer.parseInt(st.nextToken());
        int e = Integer.parseInt(st.nextToken());

        Arrays.sort(edge, (a, b) -> Integer.compare(b[2], a[2]));

        p = new int[n + 1];
        for (int[] t : edge) {
            int u = t[0], v = t[1], w = t[2];

            union(u, v);
            if (find(s) == find(e)) {
                System.out.println(w);
                return;
            }
        }
    }
}
```

직장인 파댕이의 사회생활(30985)

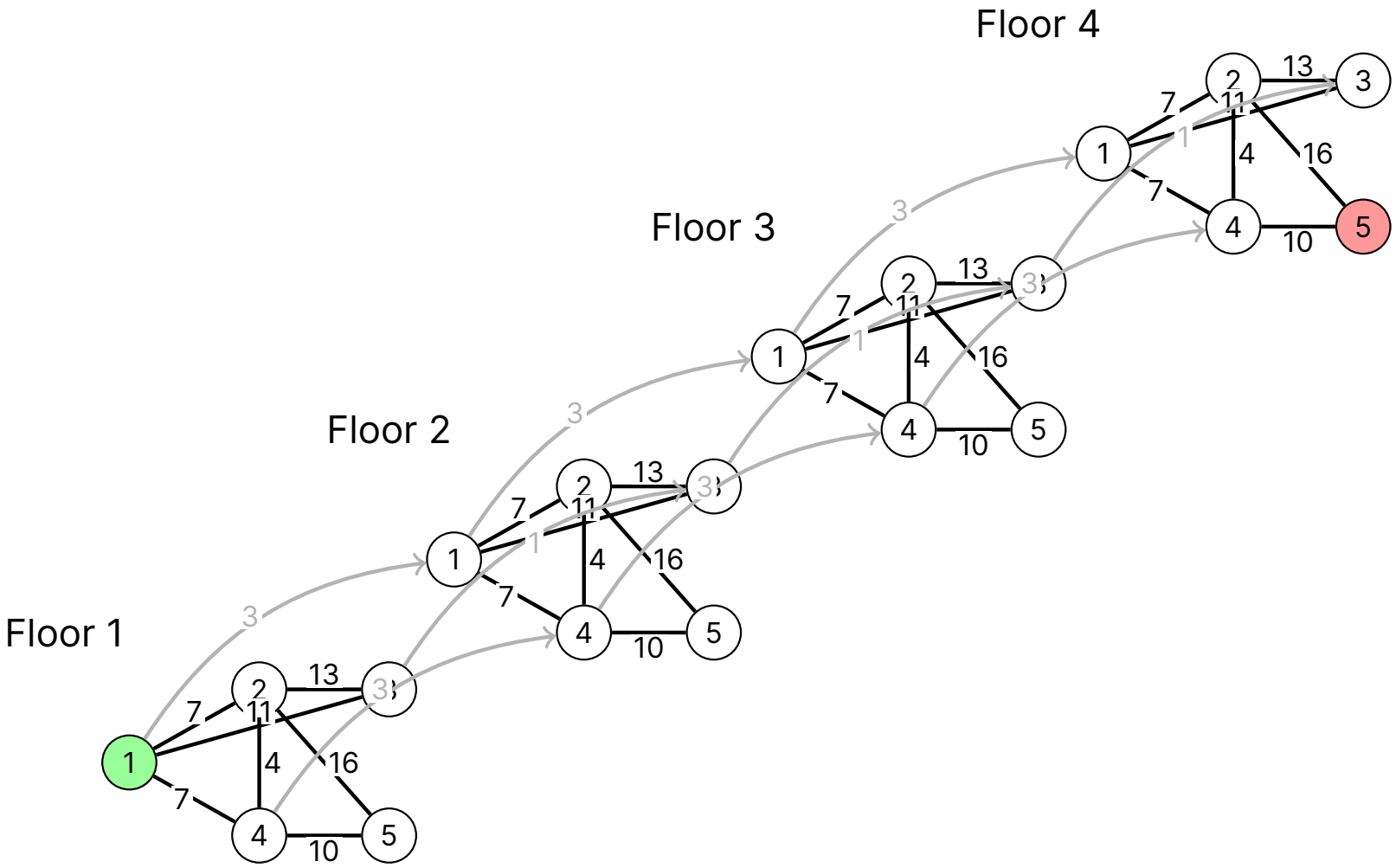
문제

파댕이는 중견기업 회사에서 직장인으로 일하고 있다. 사장님이 직장인 파댕이를 무척 아끼기 때문에, 파댕이는 사장실에 찾아가 사장님께 인사를 하려고 한다.
직장인 파댕이의 회사가 있는 건물은 K층으로 구성되어 있는데 각 층은 방과 복도로 구성되어 있다. 복도를 통해 방과 방 사이를 양방향으로 이동할 수 있다. 모든 층은 같은 모습을 하고 있다.
파댕이는 현재 1층의 1번 방에 있고, 사장실은 K층의 N번 방이다. 파댕이는 현재 자신의 위치에서부터 사장실까지 최소 시간을 사용하여 도착할 방법을 찾아야 한다.
파댕이는 각각의 복도를 지나가는 데 걸리는 시간이 얼마인지 알고 있다. 더불어 어떤 방에는 엘리베이터가 있어서 그 엘리베이터를 타고 위층으로 올라갈 수 있다. 한 층의 i번 방에 설치된 엘리베이터는 다른 층에 있는 모든 i번 방을 연결한다. 특정 엘리베이터에만 사람이 몰릴 수 있기 때문에 어떤 엘리베이터는 빠르고 어떤 엘리베이터는 느릴 수 있다. 파댕이는 각각의 엘리베이터가 한 층을 올라가는 데 걸리는 시간이 얼마인지도 알고 있다.
건물의 모습과 엘리베이터의 속도가 주어지면, 파댕이가 사장실까지 가는 최소 시간을 계산하는 프로그램을 작성하시오.

입력

첫째 줄에 방의 수 $N\,(2 \le N \le 100{,}000)$과 복도의 수 $M\,(1 \le M \le 300{,}000)$, 건물의 층수 $K\,(2 \le K \le 200{,}000)$가 공백으로 구분되어 주어진다.
다음 줄부터 M개의 줄에 걸쳐 세 정수 u, v, c가 공백으로 구분되어 주어진다.
$(1 \le u, v \le N, 1 \le c \le 100{,}000{,}000, u \ne v)$ 이는 u번 방과 v번 방을 잇는 복도가 존재하며 이를 지나가는 데 걸리는 시간이 c라는 뜻이다.
임의의 서로 다른 두 방에 대해 한쪽 방에서 다른 쪽 방으로 이어진 복도는 1개 이하다.
다음 줄에 N개의 정수 $e_1, \cdots, e_N$이 공백으로 구분되어 주어진다. $(e_i = -1$ or $1 \le e_i \le 100{,}000{,}000)$ 이는 i번째 방에 존재하는 엘리베이터를 타면 한 층을 올라갈 때마다 e_i의 시간이 든다는 뜻이다.
만약 i번째 방에 엘리베이터가 존재하지 않는다면, $e_i = -1$이다.

예제



풀이

- 엘레베이터는 반드시 1 층에서 타서 K 층까지 한 번에 간다고 생각하더라도 최적해를 구할 수 있습니다.
- 모든 엘리베이터 $e_1, e_2, \cdots, e_N$ 를 시작점으로 해서 1 번방과 N 번방 까지의 경로를 구할 수도 있습니다.
- 반대로 1 번 방과 N 번 방을 시작점으로 해서 모든 엘리베이터까지의 경로를 구할 수도 있습니다.

원찬혁 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int n, m, k;
    static List<Pair> room[];
    static int[] e;
    static boolean[][] visited;

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        PriorityQueue<Node> q = new PriorityQueue<>();
        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        k = Integer.parseInt(st.nextToken());
        room = new List[n + 1];
        e = new int[n + 1];
        visited = new boolean[n + 1][2];
        int u, v, c;
        long res = -1;
        for (int i = 1; i <= n; i++)
            room[i] = new ArrayList<>();
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            u = Integer.parseInt(st.nextToken());
            v = Integer.parseInt(st.nextToken());
            c = Integer.parseInt(st.nextToken());
            room[v].add(new Pair(u, c));
```

원찬혁 소스 코드 2

```
            room[u].add(new Pair(v, c));
        }
        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= n; i++)
            e[i] = Integer.parseInt(st.nextToken());

        q.offer(new Node(0, 1, 0));
        while (!q.isEmpty()) {
            Node node = q.poll();
            if (node.n == n && node.k == 1) {
                res = node.w;
                break;
            }
            if (visited[node.n][node.k])
                continue;

            visited[node.n][node.k] = true;
            if (node.k == 0 && e[node.n] > 0) {
                if (!visited[node.n][1])
                    q.offer(new Node(node.w + ((long) e[node.n] * (k - 1)),
                    ↪  node.n, 1));
            }
            for (Pair p : room[node.n]) {
                if (!visited[p.first][node.k])
                    q.offer(new Node(node.w + p.second, p.first, node.k));
            }
        }
        System.out.println(res);
    }
```

원찬혁 소스 코드 3

```
58  }
59
60  class Pair {
61      int first;
62      int second;
63
64      public Pair(int first, int second) {
65          this.first = first;
66          this.second = second;
67      }
68  }
69
70  class Node implements Comparable<Node> {
71      long w;
72      int n;
73      int k;
74
75      public Node(long w, int n, int k) {
76          this.w = w;
77          this.n = n;
78          this.k = k;
79      }
80
81      @Override
82      public int compareTo(Node o) {
83          return Long.compare(this.w, o.w);
84      }
85  }
```

정의찬 소스 코드 1

```java
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws NumberFormatException, IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());
        int k = Integer.parseInt(st.nextToken());

        List<int[]>[] nexts = new List[n + 1];
        int[] elev = new int[n + 1];

        for (int i = 1; i <= n; i++)
            nexts[i] = new ArrayList<>();

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());
            nexts[u].add(new int[] { v, w });
            nexts[v].add(new int[] { u, w });
        }

        st = new StringTokenizer(br.readLine());
```

정의찬 소스 코드 2

```java
        for (int i = 1; i <= n; i++) {
            elev[i] = Integer.parseInt(st.nextToken());
        }

        // 엘베타기전dist
        long[] beforeElevTime = new long[n + 1];
        // 엘베탄후dist
        long[] afterElevTime = new long[n + 1];
        Arrays.fill(beforeElevTime, Long.MAX_VALUE);
        Arrays.fill(afterElevTime, Long.MAX_VALUE);

        PriorityQueue<long[]> pq = new PriorityQueue<>((o1, o2) -> Long.compare(o1[1], o2[1]));
        pq.add(new long[] { 1, 0, 1 }); // 도착지, 걸린시간, 엘베탈수있는 기회

        while (!pq.isEmpty()) {
            long[] current = pq.poll();
            int currentN = (int) current[0];
            long currentTime = current[1];
            int chance = (int) current[2];

            if (currentN == n && chance == 0) {
                System.out.println(currentTime);
                return;
            }

            // 현위치 엘베 탈 수 있는경우
            long elevCost = (long) elev[currentN] * (k - 1);
            if (chance == 1 && elev[currentN] != -1 && (currentTime + elevCost < afterElevTime[currentN])) {
```

정의찬 소스 코드 3

```
                    afterElevTime[currentN] = currentTime + elevCost;
                    pq.add(new long[] { currentN, currentTime + elevCost, 0 });
                }
                for (int[] next : nexts[currentN]) {
                    long nextD = currentTime + next[1];
                    // 엘베 이미탄경우
                    if (chance == 0) {
                        if (nextD < afterElevTime[next[0]]) {
                            afterElevTime[next[0]] = nextD;
                            pq.add(new long[] { next[0], nextD, 0 });
                        }
                    }
                    // 엘베 안 탄경우
                    else {
                        if (nextD < beforeElevTime[next[0]]) {
                            beforeElevTime[next[0]] = nextD;
                            pq.add(new long[] { next[0], nextD, 1 });
                        }
                    }
                }
            }
            System.out.println(-1);
        }
    }
```

서민종 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int N, M, K;
    static long[] elevators;
    static Long[][] minDistances;
    static List<Room>[][] connections;

    static class Room implements Comparable<Room> {
        int number;
        int top;
        long distance;

        public Room(int number, int top, long distance) {
            this.number = number;
            this.top = top;
            this.distance = distance;
        }

        @Override
        public int compareTo(Room r) {
            if (this.distance < r.distance) {
                return -1;
            } else {
                return 1;
            }
        }
    }
```

서민종 소스 코드 2

```java

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
        StringTokenizer st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());
        K = Integer.parseInt(st.nextToken());
        connections = new List[N + 1][2];
        for (int i = 1; i <= N; i++) {
            connections[i][0] = new ArrayList<>();
            connections[i][1] = new ArrayList<>();
        }
        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            long c = Long.parseLong(st.nextToken());
            connections[u][0].add(new Room(v, 0, c));
            connections[u][1].add(new Room(v, 1, c));
            connections[v][0].add(new Room(u, 0, c));
            connections[v][1].add(new Room(u, 1, c));
        }
        st = new StringTokenizer(br.readLine());
        elevators = new long[N + 1];
        for (int i = 1; i <= N; i++) {
            elevators[i] = Long.parseLong(st.nextToken());
            if (elevators[i] >= 1) {
```

서민종 소스 코드 3

```
                connections[i][0].add(new Room(i, 1, elevators[i] * (K - 1)));
            }
        }
        long answer = dijkstra();
        bw.write(answer + "\n");
        bw.flush();
    }

    public static long dijkstra() {
        PriorityQueue<Room> pq = new PriorityQueue<>();
        boolean[][] visited = new boolean[N + 1][2];
        minDistances = new Long[N + 1][2];
        minDistances[1][0] = 0L;
        pq.offer(new Room(1, 0, 0L));
        while (!pq.isEmpty()) {
            Room currentRoom = pq.poll();
            int cn = currentRoom.number;
            int ct = currentRoom.top;
            long cd = currentRoom.distance;
            if (visited[cn][ct]) {
                continue;
            } else {
                minDistances[cn][ct] = cd;
                visited[cn][ct] = true;
                if (cn == N && ct == 1) {
                    return cd;
                }
                for (Room nextRoom : connections[cn][ct]) {
                    int nn = nextRoom.number;
```

서민종 소스 코드 4

```
                    int nt = nextRoom.top;
                    long nd = nextRoom.distance;
                    if (!visited[nn][nt]) {
                        if (minDistances[nn][nt] == null || minDistances[nn][nt] > cd + nd) {
                            pq.offer(new Room(nn, nt, cd + nd));
                            minDistances[nn][nt] = cd + nd;
                        }
                    }
                }
            }
        }
        return -1L;
    }
}
```

김준형 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static class Cord implements Comparable<Cord> {
        int n;
        long w;
        boolean elev; // 엘리베이터를 탔는지?

        public Cord(int n, long w, boolean elev) {
            this.n = n;
            this.w = w;
            this.elev = elev;
        }

        @Override
        public int compareTo(Cord c) {
            return Long.compare(this.w, c.w);
        }
    }

    static int n, m, k;
    static List<List<Cord>> graph = new ArrayList<>();
    static List<Integer> elevator = new ArrayList<>();
    static Long[] bottomTable;
    static Long[] targetTable;
    static PriorityQueue<Cord> que = new PriorityQueue<>();

    public static void main(String[] args) throws IOException {
```

김준형 소스 코드 2

```
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        k = Integer.parseInt(st.nextToken());
        bottomTable = new Long[n + 1];
        targetTable = new Long[n + 1];
        Arrays.fill(bottomTable, Long.MAX_VALUE);
        Arrays.fill(targetTable, Long.MAX_VALUE);

        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int s = Integer.parseInt(st.nextToken());
            int e = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());
            graph.get(s).add(new Cord(e, w, false));
            graph.get(e).add(new Cord(s, w, false));

        }

        elevator.add(-1);
        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= n; i++) {
            int e = Integer.parseInt(st.nextToken());
            graph.get(i).add(new Cord(i, e, false));
```

김준형 소스 코드 3

```
            elevator.add(e);

        }

        que.offer(new Cord(1, 0, false));
        bottomTable[1] = 0L;

        while (!que.isEmpty()) {
            Cord cur = que.poll();
            int curN = cur.n;
            long curW = cur.w;
            boolean curE = cur.elev;

            if (curE) {
                if (curW > targetTable[curN]) {
                    continue;
                }
            } else {
                if (curW > bottomTable[curN]) {
                    continue;
                }
            }

            for (int i = 0; i < graph.get(curN).size(); i++) {
                Cord next = graph.get(curN).get(i);
                int nextN = next.n;
                long nextW = next.w + curW;
                boolean nextE = curE;
                if (nextN == curN) {
```

김준형 소스 코드 4

```
                    if (!nextE) {
                        long elevW = elevator.get(nextN);
                        if (elevW >= 1) {
                            nextW += (elevW * (k - 2));
                            if (nextW < targetTable[nextN]) {
                                targetTable[nextN] = nextW;
                                que.offer(new Cord(nextN, nextW, true));
                            }
                        }
                    }
                } else {
                    if (nextE) { // 엘리베이터를 탔었음
                        if (nextW < targetTable[nextN]) {
                            targetTable[nextN] = nextW;
                            que.offer(new Cord(nextN, nextW, nextE));
                        }
                    } else { // 엘리베이터를 타지 않았음
                        if (nextW < bottomTable[nextN]) {
                            bottomTable[nextN] = nextW;
                            que.offer(new Cord(nextN, nextW, nextE));
                        }
                    }
                }

            }
        }
        long ans = targetTable[n] == Long.MAX_VALUE ? -1 : targetTable[n];
        System.out.print(ans);
```

김준형 소스 코드 5

```
        }
    }
```

김지훈 소스 코드 1

```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.StringTokenizer;

public class BOJ_30985 {
    static int N, M, K;
    static long[][] dist;
    static ArrayList<ArrayList<Edge>> adjList;
    static long INF = Long.MAX_dist / 2;

    public static void main(String[] args) throws Exception {

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        N = Integer.parseInt(st.nextToken()); // 정점
        M = Integer.parseInt(st.nextToken()); // 간선
        K = Integer.parseInt(st.nextToken()); // 층수

        dist = new long[2][N + 1];
        for (long[] row : dist) {
            Arrays.fill(row, INF);
        }

        adjList = new ArrayList<>();
        for (int i = 0; i <= N; i++) {
```

김지훈 소스 코드 2

```
            adjList.add(new ArrayList<>());
        }

        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            long c = Integer.parseInt(st.nextToken());
            adjList.get(u).add(new Edge(v, c));
            adjList.get(v).add(new Edge(u, c));
        }

        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= N; i++) {
            long c = Integer.parseInt(st.nextToken());
            if (c != -1) {
                adjList.get(i).add(new Edge(i, c * (K - 1)));
            }
        }

        solve();

        if (dist[1][N] == INF)
            System.out.println(-1);
        else
            System.out.println(dist[1][N]);
    }

    private static void solve() {
```

김지훈 소스 코드 3

```
        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.add(new Node(1, 0, 0));
        dist[0][1] = 0;

        while (!pq.isEmpty()) {
            Node cur = pq.poll();

            if (dist[cur.flag][cur.vertex] < cur.dist)
                continue;

            for (Edge next : adjList.get(cur.vertex)) {
                // 같은 층 이동
                if (next.to != cur.vertex) {
                    long nextDist = cur.dist + next.weight;
                    if (dist[cur.flag][next.to] > nextDist) {
                        dist[cur.flag][next.to] = nextDist;
                        pq.add(new Node(next.to, nextDist, cur.flag));
                    }
                }
                // 엘리베이터 탑승
                else if (cur.flag == 0) {
                    long nextDist = cur.dist + next.weight;
                    if (dist[1][next.to] > nextDist) {
                        dist[1][next.to] = nextDist;
                        pq.add(new Node(next.to, nextDist, 1));
                    }
                }
            }
        }
```

김지훈 소스 코드 4

```
    }
}

class Node implements Comparable<Node> {
    int vertex, flag;
    long dist;

    public Node(int vertex, long dist, int flag) {
        this.vertex = vertex;
        this.dist = dist;
        this.flag = flag;
    }

    @Override
    public int compareTo(Node e) {
        return Long.compare(this.dist, e.dist);
    }
}

class Edge {
    int to;
    long weight;

    public Edge(int to, long weight) {
        this.to = to;
        this.weight = weight;
    }
}
```

임건애 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {

    static long INF = Long.MAX_VALUE;
    static int N, M, K;
    static List<ArrayList<int[]>> graph;
    static long[] dist1, distN;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        StringTokenizer st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());
        K = Integer.parseInt(st.nextToken());

        graph = new ArrayList<>();
        for (int i = 0; i <= N; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());

```

임건애 소스 코드 2

```
            graph.get(a).add(new int[] { b, w });
            graph.get(b).add(new int[] { a, w });
        }

        dist1 = new long[N + 1];
        Arrays.fill(dist1, INF);
        dijkstra(1, dist1);

        distN = new long[N + 1];
        Arrays.fill(distN, INF);
        dijkstra(N, distN);

        // 엘베있는 방, 시간
        List<int[]> has = new ArrayList<>();
        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= N; i++) {
            int take = Integer.parseInt(st.nextToken());
            if (take != -1) {
                has.add(new int[] { i, take });
            }
        }

        long min = INF;
        for (int[] n : has) {
            int x = n[0], w = n[1];

            if (dist1[x] != INF && distN[x] != INF) {
                min = Math.min(min, dist1[x] + distN[x] + (w * (K - 1L)));
            }
```

임건애 소스 코드 3

```
        }

        System.out.println(min == INF ? -1 : min);
    }

    static void dijkstra(int start, long[] dist) {
        PriorityQueue<long[]> pq = new PriorityQueue<>((o1, o2) -> Long.compare(o1[1], o2[1]));
        pq.add(new long[] { start, 0 });
        dist[start] = 0;

        while (!pq.isEmpty()) {
            long[] current = pq.poll();
            int cur = (int) current[0];
            long weight = current[1];

            if (dist[cur] < weight)
                continue;

            for (int[] next : graph.get(cur)) {
                int n = next[0], nw = next[1];

                long cost = dist[cur] + nw;
                if (dist[n] > cost) {
                    dist[n] = cost;
                    pq.add(new long[] { n, cost });
                }

            }
```

임건애 소스 코드 4

```
87
88                  }
89          }
90
91    }
```

김시온 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static int n, m, k;
    static ArrayList<ArrayList<int[]>> g = new ArrayList<>();

    static long[] dijkstra(int s) {
        PriorityQueue<long[]> q = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
        long[] d = new long[n + 1];
        Arrays.fill(d, (long) 1e18);

        q.add(new long[] { 0, s });
        d[s] = 0;

        while (!q.isEmpty()) {
            long[] t = q.poll();
            long c = t[0];
            int u = (int) t[1];

            if (d[u] != c) {
                continue;
            }

            for (int[] edge : g.get(u)) {
                int v = edge[0], w = edge[1];

                if (d[v] > c + w) {
```

김시온 소스 코드 2

```
                    d[v] = c + w;
                    q.add(new long[] { c + w, v });
                }
            }
        }

        return d;
    }

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        k = Integer.parseInt(st.nextToken());

        for (int i = 0; i <= n; i++) {
            g.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());

            g.get(u).add(new int[] { v, w });
            g.get(v).add(new int[] { u, w });
        }
```

김시온 소스 코드 3

```java

        long[] ds = dijkstra(1), de = dijkstra(n);

        int[] e = new int[n + 1];
        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= n; i++) {
            e[i] = Integer.parseInt(st.nextToken());
        }

        long min = (long) 1e18;
        for (int i = 1; i <= n; i++) {
            if (e[i] != -1) {
                long cand = ds[i] + de[i] + (long) (k - 1) * e[i];
                min = Math.min(min, cand);
            }
        }

        System.out.println(min == (long) 1e18 ? -1 : min);
    }
}
```

소방차(25636)

문제

ANA 도시는 N개의 교차로와 M개의 양방향 도로로 이루어져 있고, 교차로는 1번부터 N번까지 번호가 매겨져 있다. S번 교차로에는 ANA 도시의 유일한 소방서가 위치하며, 무겸이는 이곳에서 소방차 운전 기사로 일하고 있다. 무겸이는 소방차를 타고 화재가 발생한 교차로에 출동해서 화재를 진압한다.
소방차에는 물을 담을 수 있는 물탱크가 있고, 이동 중에 교차로에 도착할 때마다 교차로에 설치된 소화전을 이용해 물탱크에 물을 충전한다. 소방차가 i번째 교차로에 한 번 도착할 때마다 충전하는 물의 양은 a_i 리터이다. 소방차는 소방서가 위치한 교차로에서 출동할 때와 화재가 발생한 교차로에 도착했을 때에도 물을 충전한다. 물을 충전하는 데에는 시간이 걸리지 않으며, 물탱크의 용량에도 제한이 없다.
어느 날 T번 교차로에 화재가 발생했다. 무겸이는 소방차를 타고 S번 교차로에서 T번 교차로까지 **최단 경로**로 이동한다. 단, 최단 경로는 여러 개일 수 있다. 이때 무겸이는 여러 최단 경로 중에서 **물탱크에 담긴 물의 양이 최대가 되는 경로**를 선택한다. 무겸이가 T번 교차로에 도착했을 때 소방차의 이동 거리와 물탱크에 충전된 물의 양을 구하시오.

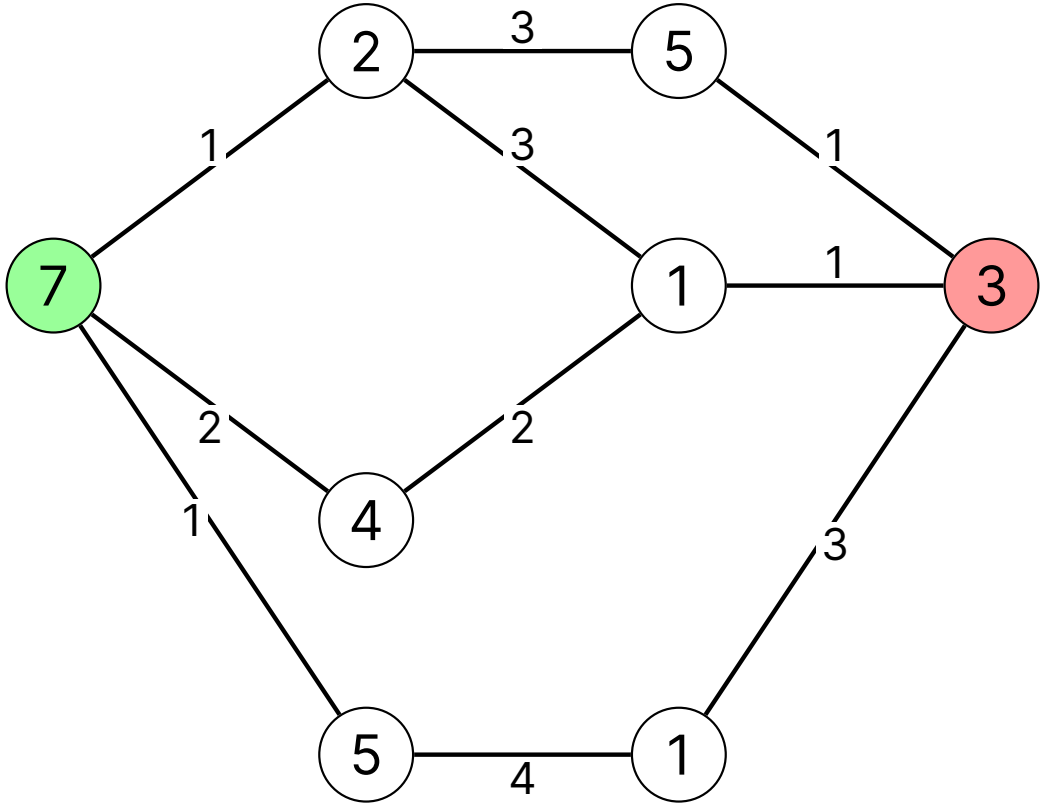
입력

첫째 줄에 교차로의 수 N $(2 \le N \le 100\,000)$이 주어진다.
둘째 줄에 $a_1, a_2, \dots, a_N$이 주어진다. $(1 \le a_i \le 1\,000\,000)$
셋째 줄에 도로의 수 M $(1 \le M \le 300\,000)$이 주어진다.
다음 M개의 줄에 도로 정보 u, v, w가 주어진다. $(1 \le u, v \le N,\ u \ne v,\ 1 \le w \le 1\,000\,000)$
마지막 줄에 소방서 위치 S와 화재 발생 교차로 T가 주어진다. $(1 \le S, T \le N,\ S \ne T)$

출력

S번 교차로에서 T번 교차로까지의 **최단 거리**와, 그 최단 경로들 중에서 **가장 많은 물을 모을 수 있는 경로의 물의 양**을 출력한다. 만약 T번 교차로에 도달할 수 없다면 -1만 출력한다.

예시 그래프



원찬혁 소스 코드 1

```
1   import java.util.*;
2   import java.io.*;
3
4   public class Main {
5       static int n, m;
6       static int a[];
7       static boolean visited[];
8       static List<Pair> e[];
9
10      public static void main(String[] args) throws Exception {
11          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
12          StringTokenizer st;
13          n = Integer.parseInt(br.readLine());
14          int u, v, w;
15          a = new int[n + 1];
16          visited = new boolean[n + 1];
17          st = new StringTokenizer(br.readLine());
18          for (int i = 1; i <= n; i++)
19              a[i] = Integer.parseInt(st.nextToken());
20          m = Integer.parseInt(br.readLine());
21          e = new List[n + 1];
22          for (int i = 1; i <= n; i++)
23              e[i] = new ArrayList<>();
24          for (int i = 0; i < m; i++) {
25              st = new StringTokenizer(br.readLine());
26              u = Integer.parseInt(st.nextToken());
27              v = Integer.parseInt(st.nextToken());
28              w = Integer.parseInt(st.nextToken());
29              e[u].add(new Pair(v, w));
```

원찬혁 소스 코드 2

```
30                e[v].add(new Pair(u, w));
31            }
32            st = new StringTokenizer(br.readLine());
33            u = Integer.parseInt(st.nextToken());
34            v = Integer.parseInt(st.nextToken());
35            PriorityQueue<Node> pq = new PriorityQueue<>();
36            pq.offer(new Node(u, 0, a[u]));
37            long level = 0, res = -1;
38            while (!pq.isEmpty()) {
39                Node node = pq.poll();
40                if (node.v == v) {
41                    level = node.w;
42                    res = node.a;
43                    break;
44                }
45                if (visited[node.v])
46                    continue;
47                visited[node.v] = true;
48                for (Pair p : e[node.v]) {
49                    if (!visited[p.v])
50                        pq.offer(new Node(p.v, p.w + node.w, a[p.v] + node.a));
51                }
52            }
53            if (res > 0)
54                System.out.println(level + " " + res);
55            else
56                System.out.println(-1);
57        }
58    }
```

원찬혁 소스 코드 3

```java

class Pair {
    int v;
    long w;

    public Pair(int v, long w) {
        this.v = v;
        this.w = w;
    }
}

class Node implements Comparable<Node> {
    int v;
    long w;
    long a;

    public Node(int v, long w, long a) {
        this.v = v;
        this.w = w;
        this.a = a;
    }

    @Override
    public int compareTo(Node o) {
        int ret = Long.compare(this.w, o.w);
        return (ret != 0) ? ret : Long.compare(o.a, this.a);
    }
}
```

정의찬 소스 코드 1

```java
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws NumberFormatException, IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        StringTokenizer st;
        int n = Integer.parseInt(br.readLine());
        int[] water = new int[n + 1];
        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= n; i++) {
            water[i] = Integer.parseInt(st.nextToken());
        }
        List<int[]>[] nexts = new List[n + 1];
        for (int i = 1; i <= n; i++)
            nexts[i] = new ArrayList<>();
        int m = Integer.parseInt(br.readLine());
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());
            nexts[u].add(new int[] { v, w });
            nexts[v].add(new int[] { u, w });
        }
        st = new StringTokenizer(br.readLine());
        int s = Integer.parseInt(st.nextToken());
```

정의찬 소스 코드 2

```java
        int e = Integer.parseInt(st.nextToken());

        long[] maxWater = new long[n + 1];
        long[] dist = new long[n + 1];
        Arrays.fill(dist, Long.MAX_VALUE);

        PriorityQueue<long[]> pq = new PriorityQueue<>(
                (o1, o2) -> o1[1] == o2[1] ? Long.compare(o2[2], o1[2]) :
                ↪  Long.compare(o1[1], o2[1]));
        // 도착지, 거리, 물
        pq.add(new long[] { s, 0, water[s] });

        while (!pq.isEmpty()) {
            long[] current = pq.poll();
            int currentN = (int) current[0];
            long currentDist = (long) current[1];
            long currentWater = (long) current[2];

            if (currentWater <= maxWater[currentN])
                continue;
            maxWater[currentN] = currentWater;

            if (currentN == e) {
                System.out.println(currentDist + " " + currentWater);
                return;
            }

            for (int[] next : nexts[currentN]) {
                long nextD = currentDist + next[1];
```

정의찬 소스 코드 3

```
                    long nextWater = currentWater + water[next[0]];
                    if (nextD <= dist[next[0]]) {
                        dist[next[0]] = nextD;
                        pq.add(new long[] { next[0], nextD, nextWater });
                    }
                }
            }

            System.out.println(-1);
        }
    }
```

서민종 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int N, M, S, T;
    static long[] water;
    static HashMap<Integer, Long>[] roads;
    static Long[] minDistances;
    static Long[] maxWater;

    static class Road implements Comparable<Road> {
        int num;
        long distance;
        long water;

        public Road(int num, long distance, long water) {
            this.num = num;
            this.distance = distance;
            this.water = water;
        }

        @Override
        public int compareTo(Road r) {
            if (this.distance != r.distance) {
                if (this.distance < r.distance) {
                    return -1;
                }
                return 1;
            } else {
```

서민종 소스 코드 2

```
                if (this.water > r.water) {
                    return -1;
                } else {
                    return 1;
                }
            }
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new
        ↪  OutputStreamWriter(System.out));
        StringBuilder sb = new StringBuilder();
        N = Integer.parseInt(br.readLine());
        water = new long[N + 1];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= N; i++) {
            water[i] = Long.parseLong(st.nextToken());
        }
        roads = new HashMap[N + 1];
        for (int i = 1; i <= N; i++) {
            roads[i] = new HashMap<>();
        }
        M = Integer.parseInt(br.readLine());
        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
```

서민종 소스 코드 3

```
            int w = Integer.parseInt(st.nextToken());
            roads[u].put(v, Math.min(w, roads[u].getOrDefault(v, 1000000L)));
            roads[v].put(u, Math.min(w, roads[v].getOrDefault(u, 1000000L)));
        }
        st = new StringTokenizer(br.readLine());
        S = Integer.parseInt(st.nextToken());
        T = Integer.parseInt(st.nextToken());
        dijkstra();
        if (minDistances[T] == null) {
            sb.append(-1);
        } else {
            sb.append(minDistances[T]).append(" ").append(maxWater[T]);
        }
        sb.append("\n");
        bw.write(sb.toString());
        bw.flush();
    }

    public static void dijkstra() {
        minDistances = new Long[N + 1];
        maxWater = new Long[N + 1];
        boolean[] visited = new boolean[N + 1];
        PriorityQueue<Road> pq = new PriorityQueue<>();
        visited[S] = true;
        minDistances[S] = 0L;
        maxWater[S] = water[S];
        for (int num : roads[S].keySet()) {
            long distance = roads[S].get(num);
            pq.offer(new Road(num, distance, water[S] + water[num]));
```

서민종 소스 코드 4

```
                minDistances[num] = distance;
                maxWater[num] = water[S] + water[num];
            }
            while (!pq.isEmpty()) {
                Road current = pq.poll();
                int cn = current.num;
                long cd = current.distance;
                long cw = current.water;
                if (!visited[cn]) {
                    visited[cn] = true;
                    for (int nn : roads[cn].keySet()) {
                        long nd = roads[cn].get(nn);
                        long nw = water[nn];
                        if (!visited[nn]) {
                         if (minDistances[nn] == null || minDistances[nn] > cd + nd) {
                                minDistances[nn] = cd + nd;
                                maxWater[nn] = cw + nw;
                                pq.offer(new Road(nn, cd + nd, cw + nw));
                           } else if (minDistances[nn] == cd + nd && maxWater[nn] < cw
                             ↪  + nw) {
                                maxWater[nn] = cw + nw;
                                pq.offer(new Road(nn, cd + nd, cw + nw));
                            }
                        }
                    }
                }
            }
        }
    }
```

임건애 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {

    static int N, M;
    static List<ArrayList<int[]>> graph;
    static int[] water;
    static long[] dist, sum;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        N = Integer.parseInt(br.readLine());

        StringTokenizer st = new StringTokenizer(br.readLine());
        water = new int[N + 1];
        for (int i = 1; i <= N; i++) {
            water[i] = Integer.parseInt(st.nextToken());
        }

        graph = new ArrayList<>();
        for (int i = 0; i <= N; i++) {
            graph.add(new ArrayList<>());
        }

        M = Integer.parseInt(br.readLine());
        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
```

임건애 소스 코드 2

```
30                int u = Integer.parseInt(st.nextToken());
31                int v = Integer.parseInt(st.nextToken());
32                int w = Integer.parseInt(st.nextToken());
33
34                graph.get(u).add(new int[] { v, w });
35                graph.get(v).add(new int[] { u, w });
36            }
37
38            st = new StringTokenizer(br.readLine());
39            int S = Integer.parseInt(st.nextToken());
40            int T = Integer.parseInt(st.nextToken());
41
42            dist = new long[N + 1];
43            Arrays.fill(dist, Long.MAX_VALUE);
44            sum = new long[N + 1];
45            dijkstra(S);
46
47            StringBuilder sb = new StringBuilder();
48            if (dist[T] == Long.MAX_VALUE) {
49                sb.append(-1);
50            } else {
51                sb.append(dist[T]).append(" ").append(sum[T]);
52            }
53
54            System.out.println(sb);
55
56        }
57
58        static void dijkstra(int start) {
```

임건애 소스 코드 3

```java
        PriorityQueue<long[]> pq = new PriorityQueue<>((o1, o2) -> {
            if (o1[1] == o2[1]) {
                return Long.compare(o2[2], o1[2]); // dist가 같다면 물 양 내림차순
            }
            return Long.compare(o1[1], o2[1]);
        });
        pq.add(new long[] { start, 0, water[start] });
        dist[start] = 0;
        sum[start] = water[start];

        while (!pq.isEmpty()) {
            long[] current = pq.poll();
            int cur = (int) current[0];
            long weight = current[1], wtr = current[2];

            if (dist[cur] < weight || (dist[cur] == weight && sum[cur] > wtr))
                continue;

            for (int[] next : graph.get(cur)) {
                int n = next[0], nw = next[1];

                long cost = dist[cur] + nw;
                long sumWater = wtr + water[n];

                // dist가 더 크거나, 같을 때 sumWater가 더 작다면
                if (dist[n] > cost || (dist[n] == cost && sum[n] < sumWater)) {
                    dist[n] = cost;
                    sum[n] = sumWater;
                    pq.add(new long[] { n, cost, sumWater });
```

임건애 소스 코드 4

```
88                      }
89
90                  }
 91
92              }
93
94          }
95
96    }
```

김시온 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine());

        int[] a = new int[n + 1];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= n; i++) {
            a[i] = Integer.parseInt(st.nextToken());
        }

        ArrayList<ArrayList<int[]>> g = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            g.add(new ArrayList<>());
        }

        int m = Integer.parseInt(br.readLine());
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());

            g.get(u).add(new int[] { v, w });
            g.get(v).add(new int[] { u, w });
        }
```

김시온 소스 코드 2

```java

        st = new StringTokenizer(br.readLine());
        int s = Integer.parseInt(st.nextToken());
        int t = Integer.parseInt(st.nextToken());

        long[][] d = new long[n + 1][2];
        for (int i = 1; i <= n; i++) {
            d[i][0] = (long) 1e18;
            d[i][1] = (long) 1e18;
        }

        PriorityQueue<long[]> q = new PriorityQueue<>((x, y) -> {
            if (x[0] != y[0]) {
                return Long.compare(x[0], y[0]);
            } else {
                return Long.compare(x[1], y[1]);
            }
        });

        d[s][0] = 0;
        d[s][1] = -a[s];

        q.add(new long[] { 0, -a[s], s });

        while (!q.isEmpty()) {
            long[] p = q.poll();

            long c1 = p[0], c2 = p[1];
            int u = (int) p[2];
```

김시온 소스 코드 3

```java

            if (d[u][0] != c1 || d[u][1] != c2) {
                continue;
            }

            for (int[] e : g.get(u)) {
                int v = e[0], w = e[1];

                long nc1 = c1 + w, nc2 = c2 - a[v];
                if (d[v][0] > nc1) {
                    d[v][0] = nc1;
                    d[v][1] = nc2;
                    q.add(new long[] { nc1, nc2, v });
                } else if (d[v][0] == nc1 && d[v][1] > nc2) {
                    d[v][0] = nc1;
                    d[v][1] = nc2;
                    q.add(new long[] { nc1, nc2, v });
                }
            }
        }

        if (d[t][0] == (long) 1e18) {
            System.out.println(-1);
        } else {
            System.out.println(d[t][0] + " " + (-d[t][1]));
        }
    }
}
```

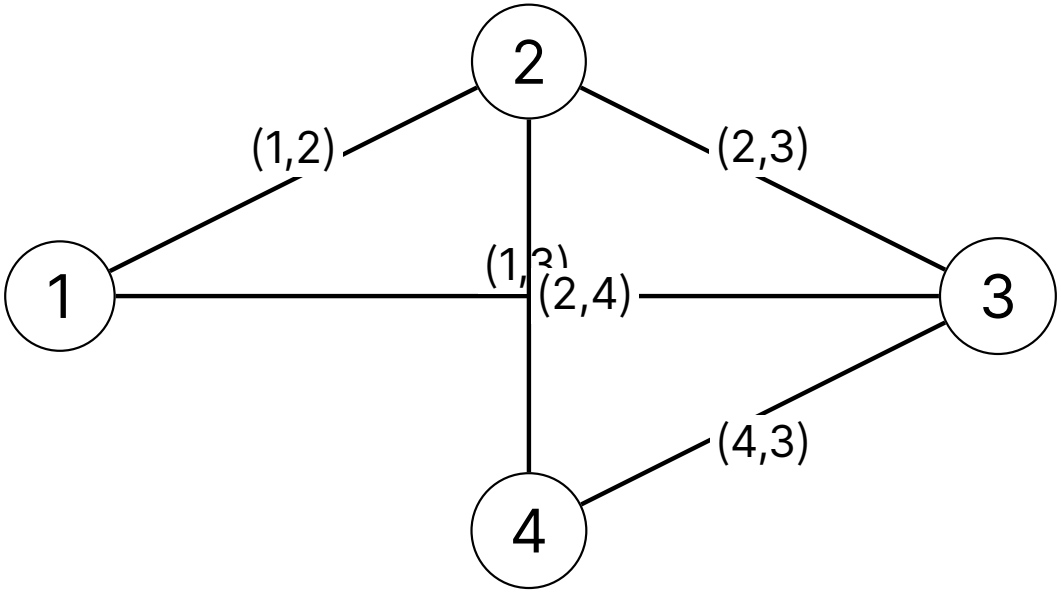
소 운전한다.(19701)

문제

소들은 날아다니는 대신 그냥 운전을 하기로 했다. 대한민국에는 도시가 V개 있고, 두 도시를 양방향으로 잇는 고속도로가 E개 있다. 모든 소들은 선린인터넷고가 있는 1번 도시에 산다. 모든 고속도로의 가운데에는 돈까스를 파는 휴게소가 하나씩 있다.
베시는 다른 도시로 여행을 떠나려 한다. 경로를 잘 골라 1번 도시에서 각 도시까지 가는 데 걸리는 시간의 최솟값을 구하려 한다. 즉, 2번부터 V번까지 각 도시까지의 최단 시간을 구하라.
하지만 베시는 휴게소의 돈까스를 먹고 싶어졌다. 각 고속도로의 휴게소에는 돈까스의 맛이 수치로 주어진다.
베시는 여행 경로 중 **정확히 한 개의 휴게소**에 들러 돈까스를 먹을 것이다.
각 도시 i에 대해 다음 값을 최소화하라.

$$(\text{1번 도시에서 i번 도시까지 이동 시간}) - (\text{먹은 돈까스의 맛})$$

예제 그래프



정의찬 소스 코드 1

```java
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws NumberFormatException, IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        StringTokenizer st = new StringTokenizer(br.readLine());

        int V = Integer.parseInt(st.nextToken());
        int E = Integer.parseInt(st.nextToken());

        List<int[]>[] nexts = new List[V + 1];
        for (int i = 1; i <= V; i++)
            nexts[i] = new ArrayList<>();
        for (int i = 0; i < E; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());
            int t = Integer.parseInt(st.nextToken());
            nexts[u].add(new int[] { v, w, t });
            nexts[v].add(new int[] { u, w, t });
        }

        long[][] dist = new long[2][V + 1];
        // 이미 먹었을때 dist
        Arrays.fill(dist[0], Long.MAX_VALUE);
```

정의찬 소스 코드 2

```java
        // 먹기 전 dist
        Arrays.fill(dist[1], Long.MAX_VALUE);

        PriorityQueue<long[]> pq = new PriorityQueue<>((o1, o2) -> Long.compare(o1[1], o2[1]));
        // 도착점, 거리, 먹을수있는횟수
        pq.add(new long[] { 1, 0, 1 });
        while (!pq.isEmpty()) {
            long[] current = pq.poll();
            int currentN = (int) current[0];
            long distance = current[1];
            int chance = (int) current[2];

            if (dist[chance][currentN] < distance)
                continue;

            for (int[] next : nexts[currentN]) {
                long nextD = distance + next[1];
                if (chance == 1) {
                    if (dist[0][next[0]] > nextD - next[2]) {
                        dist[0][next[0]] = nextD - next[2];
                        pq.add(new long[] { next[0], nextD - next[2], 0 });
                    }
                }

                if (dist[chance][next[0]] > nextD) {
                    dist[chance][next[0]] = nextD;
                    pq.add(new long[] { next[0], nextD, chance });
                }
```

정의찬 소스 코드 3

```
            }
        }

        for (int i = 2; i <= V; i++)
            sb.append(dist[0][i]).append("\n");

        System.out.println(sb);
    }
}
```

서민종 소스 코드 1

```java
package BOJ19701;

import java.util.*;
import java.io.*;

public class Main {
    static int V, E;

    static class Connection {
        int num;
        int katsu;
        long distance;

        public Connection(int num, int katsu, long distance) {
            this.num = num;
            this.katsu = katsu;
            this.distance = distance;
        }
    }

    static List<Connection>[][] connections;
    static Long[][] minDistances;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new
        ↪  OutputStreamWriter(System.out));
        StringTokenizer st = new StringTokenizer(br.readLine());
        V = Integer.parseInt(st.nextToken());
```

서민종 소스 코드 2

```
        E = Integer.parseInt(st.nextToken());
        connections = new List[V + 1][2];
        for (int i = 1; i <= V; i++) {
            connections[i][0] = new ArrayList<>();
            connections[i][1] = new ArrayList<>();
        }
        for (int i = 0; i < E; i++) {
            st = new StringTokenizer(br.readLine());
            int x = Integer.parseInt(st.nextToken());
            int y = Integer.parseInt(st.nextToken());
            long t = Long.parseLong(st.nextToken());
            long k = Long.parseLong(st.nextToken());
            connections[x][0].add(new Connection(y, 0, t));
            connections[x][0].add(new Connection(y, 1, t - k));
            connections[x][1].add(new Connection(y, 1, t));
            connections[y][0].add(new Connection(x, 0, t));
            connections[y][0].add(new Connection(x, 1, t - k));
            connections[y][1].add(new Connection(x, 1, t));
        }
        dijkstra();
        for (int i = 2; i <= V; i++) {
            bw.write(minDistances[i][1] + "\n");
        }
        bw.flush();
    }

    public static void dijkstra() {
        boolean[][] visited = new boolean[V + 1][2];
        minDistances = new Long[V + 1][2];
```

서민종 소스 코드 3

```
        minDistances[1][0] = 0L;
        PriorityQueue<Connection> pq = new PriorityQueue<>(new Comparator<Connection>() {
            @Override
            public int compare(Connection c1, Connection c2) {
                if (c1.katsu != c2.katsu) {
                    return c1.katsu - c2.katsu;
                } else {
                    if (c1.distance < c2.distance) {
                        return -1;
                    }
                    return 1;
                }
            }
        });
        for (Connection c : connections[1][0]) {
            minDistances[c.num][c.katsu] = c.distance;
            pq.offer(c);
        }
        while (!pq.isEmpty()) {
            Connection current = pq.poll();
            int cn = current.num;
            int ck = current.katsu;
            long cd = current.distance;
            if (!visited[cn][ck]) {
                visited[cn][ck] = true;
                for (Connection next : connections[cn][ck]) {
                    int nn = next.num;
                    int nk = next.katsu;
```

서민종 소스 코드 4

```
                        long nd = next.distance;
                        if (!visited[nn][nk]) {
                            if (minDistances[nn][nk] == null || minDistances[nn][nk] >
                             ↪  minDistances[cn][ck] + nd) {
                                minDistances[nn][nk] = minDistances[cn][ck] + nd;
                              pq.offer(new Connection(nn, nk, minDistances[nn][nk]));
                            }
                        }
                    }
                }
            }
        }
    }
```

원찬혁1 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main1 {
    static int n, m;
    static long res[][];
    static boolean visited[][];
    static List<Edge> e[];

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        StringBuilder sb = new StringBuilder();
        n = Integer.parseInt(st.nextToken());
        int u, v, w, k;
        res = new long[n + 1][2];
        visited = new boolean[n + 1][2];
        m = Integer.parseInt(st.nextToken());
        e = new List[n + 1];
        for (int i = 1; i <= n; i++) {
            e[i] = new ArrayList<>();
            res[i][0] = Long.MAX_VALUE;
            res[i][1] = Long.MAX_VALUE;
        }
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            u = Integer.parseInt(st.nextToken());
            v = Integer.parseInt(st.nextToken());
            w = Integer.parseInt(st.nextToken());
```

원찬혁1 소스 코드 2

```
            k = Integer.parseInt(st.nextToken());
            e[u].add(new Edge(v, w, k));
            e[v].add(new Edge(u, w, k));
        }

        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.offer(new Node(1, 0, 0, 0));
        res[1][0] = 0;
        while (!pq.isEmpty()) {
            Node node = pq.poll();
            if (visited[node.v][node.v2])
                continue;
            visited[node.v][node.v2] = true;
            for (Edge edge : e[node.v]) {
                if (res[edge.v][node.v2] > node.w + edge.w - node.k) {
                    res[edge.v][node.v2] = node.w + edge.w - node.k;
                    pq.offer(new Node(edge.v, node.v2, node.w + edge.w, node.k));
                }
                if (node.v2 == 0) {
                    if (res[edge.v][1] > node.w + edge.w - edge.k) {
                        res[edge.v][1] = node.w + edge.w - edge.k;
                        pq.offer(new Node(edge.v, 1, node.w + edge.w, edge.k));
                    }
                }
            }
        }
        for (int i = 2; i <= n; i++)
            sb.append(res[i][1]).append("\n");
        System.out.println(sb);
```

원찬혁1 소스 코드 3

```
    }
}

class Edge {
    int v;
    long w;
    int k;

    public Edge(int v, long w, int k) {
        this.v = v;
        this.w = w;
        this.k = k;
    }
}

class Node implements Comparable<Node> {
    int v;
    int v2;
    long w;
    int k;

    public Node(int v, int v2, long w, int k) {
        this.v = v;
        this.v2 = v2;
        this.w = w;
        this.k = k;
    }

    @Override
```

원찬혁1 소스 코드 4

```
    public int compareTo(Node o) {
        int ret = Integer.compare(this.v2, o.v2);
        return (ret != 0) ? ret : Long.compare(this.w - this.k, o.w - o.k);
    }
}
```

원찬혁2 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main2 {
    static int n, m;
    static long res[][];
    static List<Edge> e[];

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        StringBuilder sb = new StringBuilder();
        n = Integer.parseInt(st.nextToken());
        int u, v, w, k;
        res = new long[n + 1][2];
        m = Integer.parseInt(st.nextToken());
        e = new List[n + 1];
        for (int i = 1; i <= n; i++) {
            e[i] = new ArrayList<>();
            res[i][0] = Long.MAX_VALUE;
            res[i][1] = Long.MAX_VALUE;
        }
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            u = Integer.parseInt(st.nextToken());
            v = Integer.parseInt(st.nextToken());
            w = Integer.parseInt(st.nextToken());
            k = Integer.parseInt(st.nextToken());
            e[u].add(new Edge(v, w, k));
```

원찬혁2 소스 코드 2

```
            e[v].add(new Edge(u, w, k));
        }

        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.offer(new Node(1, 0, 0));
        res[1][0] = 0;
        while (!pq.isEmpty()) {
            Node node = pq.poll();
            int v2 = (node.k > 0) ? 1 : 0;
            if (res[node.v][v2] < node.w - node.k)
                continue;
            for (Edge edge : e[node.v]) {
                if (res[edge.v][v2] > node.w + edge.w - node.k) {
                    res[edge.v][v2] = node.w + edge.w - node.k;
                    pq.offer(new Node(edge.v, (long) node.w + edge.w, node.k));
                }
                if (v2 == 0) {
                    if (res[edge.v][1] > node.w + edge.w - edge.k) {
                        res[edge.v][1] = node.w + edge.w - edge.k;
                       pq.offer(new Node(edge.v, (long) node.w + edge.w, edge.k));
                    }
                }
            }
        }
        for (int i = 2; i <= n; i++)
            sb.append(res[i][1]).append("\n");
        System.out.println(sb);
    }
}
```

원찬혁2 소스 코드 3

```java

class Edge {
    int v;
    int w;
    int k;

    public Edge(int v, int w, int k) {
        this.v = v;
        this.w = w;
        this.k = k;
    }
}

class Node implements Comparable<Node> {
    int v;
    long w;
    int k;

    public Node(int v, long w, int k) {
        this.v = v;
        this.w = w;
        this.k = k;
    }

    @Override
    public int compareTo(Node o) {
        int ret = Integer.compare((this.k == 0) ? 0 : 1000000001,
                (o.k == 0) ? 0 : 1000000001);
        return (ret != 0) ? ret : Long.compare(this.w - this.k, o.w - o.k);
```

원찬혁2 소스 코드 4

```
88        }
89    }
```

김시온 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static final long inf = (long) 1e18;
    static final long base = 1_000_000_000L;

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());
        ArrayList<ArrayList<int[]>> g = new ArrayList<>();

        for (int i = 0; i <= n; i++) {
            g.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int t = Integer.parseInt(st.nextToken());
            int k = Integer.parseInt(st.nextToken());

            g.get(u).add(new int[] { v, t, k });
            g.get(v).add(new int[] { u, t, k });
        }
```

김시온 소스 코드 2

```java

        long[][] d = new long[n + 1][2];
        for (int i = 1; i <= n; i++) {
            d[i][0] = inf;
            d[i][1] = inf;
        }

        PriorityQueue<long[]> pq = new PriorityQueue<>(
                (a, b) -> Long.compare(a[0], b[0]));

        d[1][0] = 0;
        pq.offer(new long[] { 0, 1, 0 });

        while (!pq.isEmpty()) {
            long[] cur = pq.poll();

            long dist = cur[0];
            int u = (int) cur[1];
            int b = (int) cur[2];

            if (d[u][b] < dist) {
                continue;
            }

            for (int[] e : g.get(u)) {
                int v = e[0], t = e[1], k = e[2];

                if (d[v][b] > dist + t) {
                    d[v][b] = dist + t;
```

김시온 소스 코드 3

```
                    pq.offer(new long[] { dist + t, v, b });
                }

                if (b == 0) {
                    long nd = dist + t - k + base;
                    if (d[v][1] > nd) {
                        d[v][1] = nd;
                        pq.offer(new long[] { nd, v, 1 });
                    }
                }
            }
        }

        StringBuilder sb = new StringBuilder();
        for (int i = 2; i <= n; i++) {
            sb.append(d[i][1] - base).append('\n');
        }
        System.out.print(sb);
    }
}
```